

Text Representation

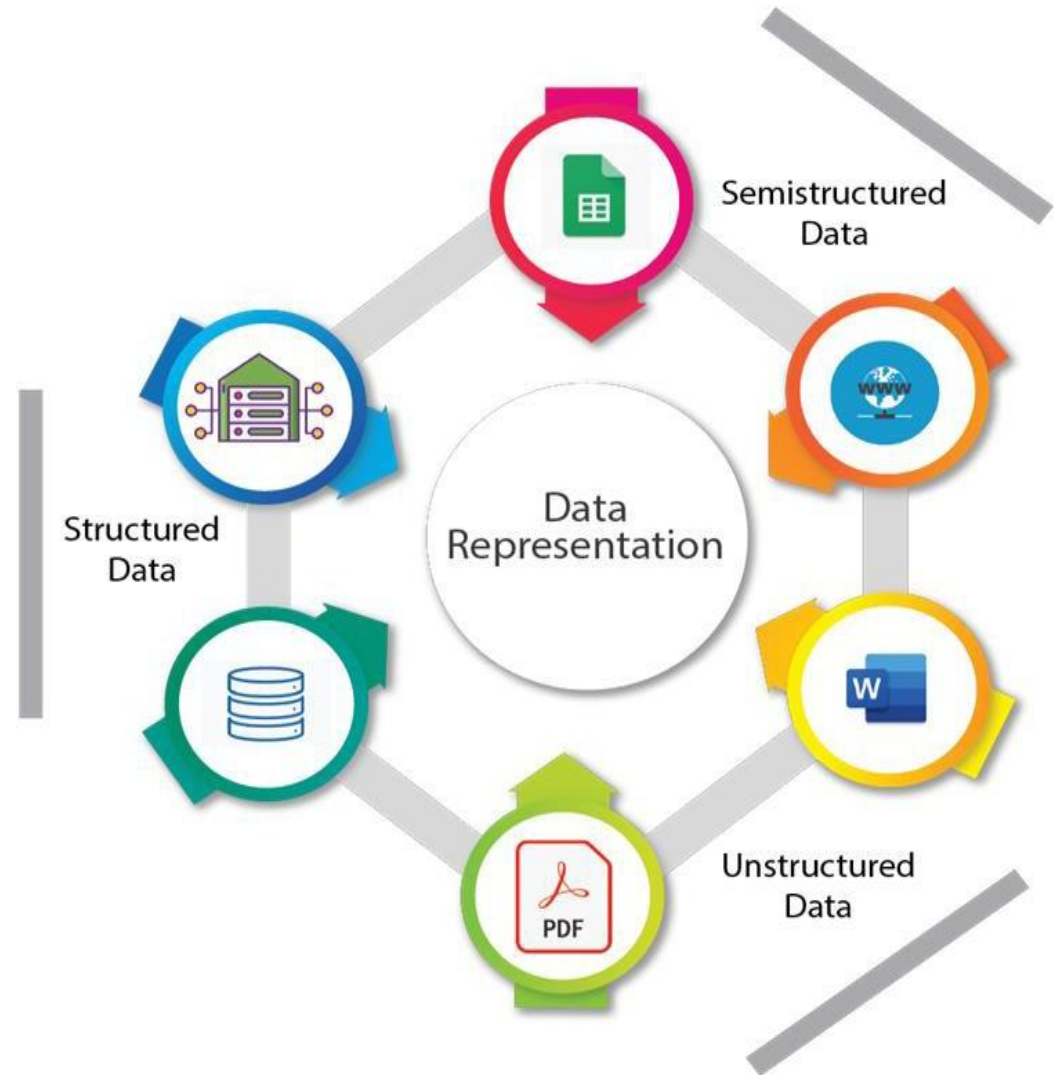
(Natural Language Processing)

Randil Pushpananda

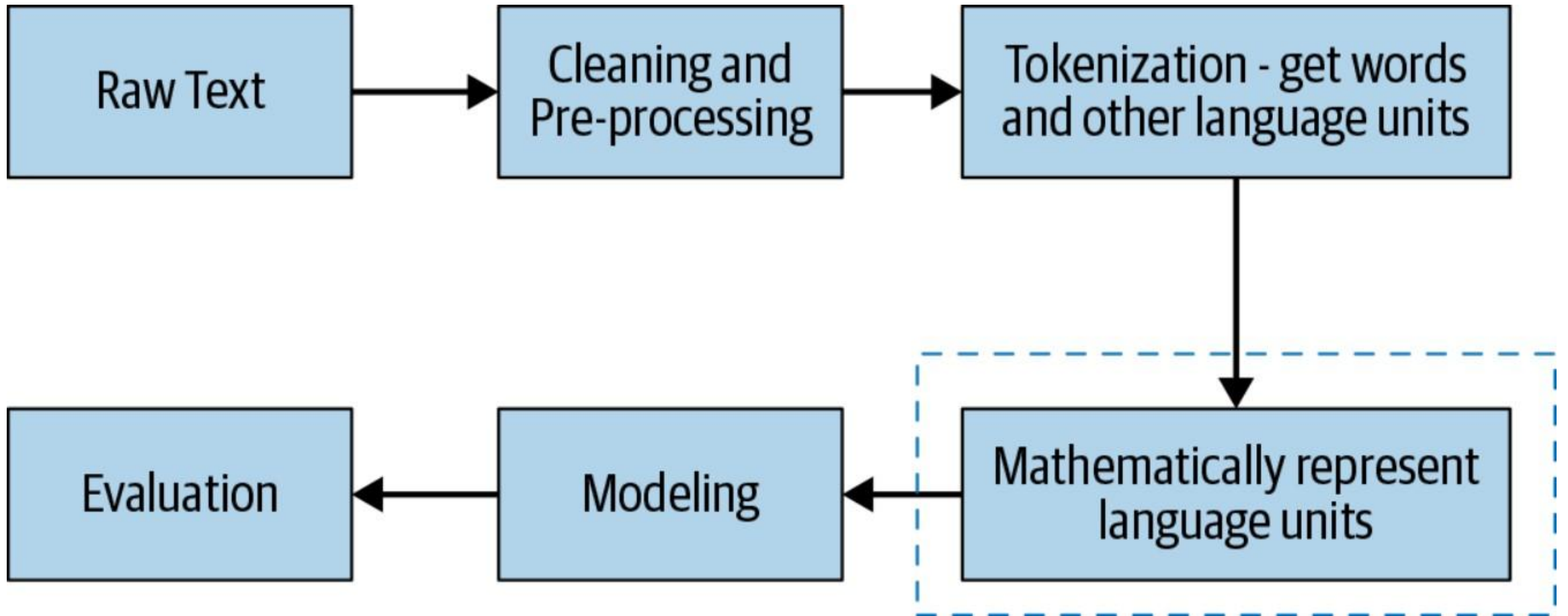
rpn@ucsc.cmb.ac.lk



TEXT REPRESENTATION



NLP Pipeline



Sound Wave Representation

Time (s)	Amplitude
0.000000	120
0.0000625	145
0.000125	132
0.0001875	98
...	

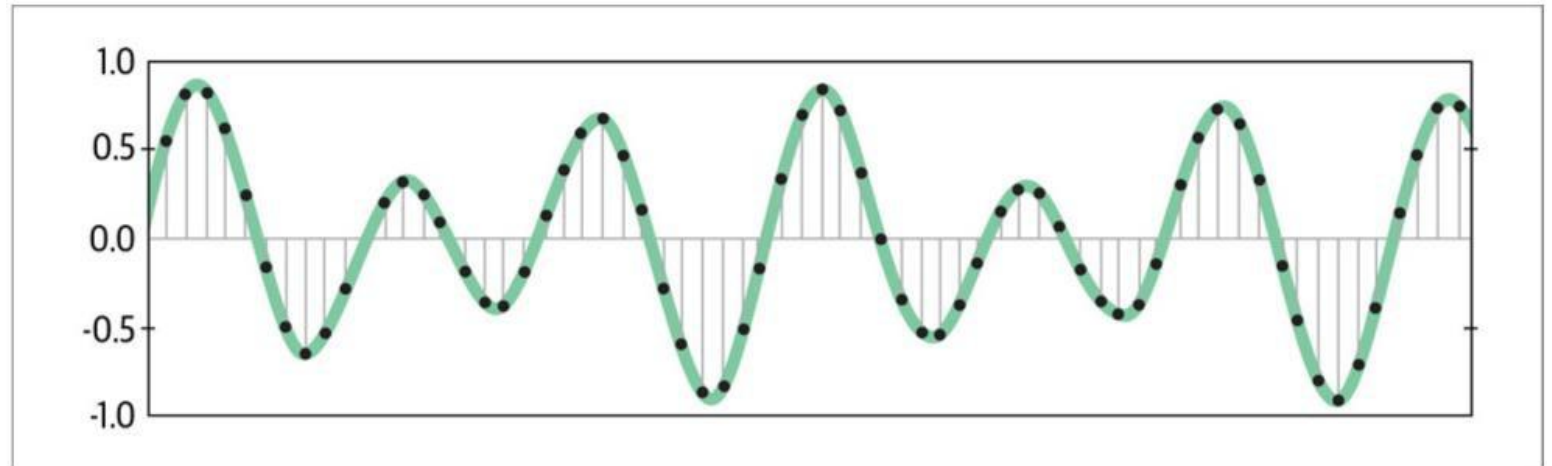


Figure 3-3. Sampling a speech wave

```
[-1274, -1252, -1160, -986, -792, -692, -614, -429, -286, -134, -57, -41,  
-169, -456, -450, -541, -761, -1067, -1231, -1047, -952, -645, -489, -448,  
-397, -212, 193, 114, -17, -110, 128, 261, 198, 390, 461, 772, 948, 1451,  
1974, 2624, 3793, 4968, 5939, 6057, 6581, 7302, 7640, 7223, 6119, 5461,  
4820, 4353, 3611, 2740, 2004, 1349, 1178, 1085, 901, 301, -262, -499,  
-488, -707, -1406, -1997, -2377, -2494, -2605, -2675, -2627, -2500, -2148,  
-1648, -970, -364, 13, 260, 494, 788, 1011, 938, 717, 507, 323, 324, 325,  
350, 103, -113, 64, 176, 93, -249, -461, -606, -909, -1159, -1307, -1544]
```

Figure 3-4. Speech signal represented by a numerical vector

Image Representation



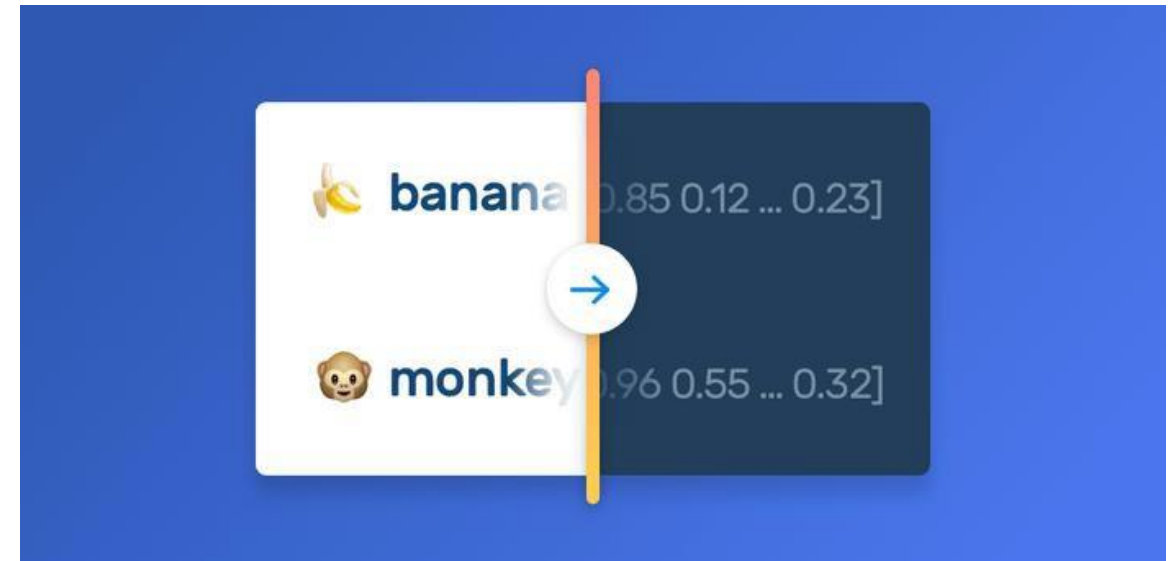
What We See

08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	08
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	48	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	53	88	30	03	49	13	36	65
52	70	95	23	04	60	11	42	69	24	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	63	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
88	36	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	38	25	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	62	99	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	86	81	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	19	67	48

What Computers See

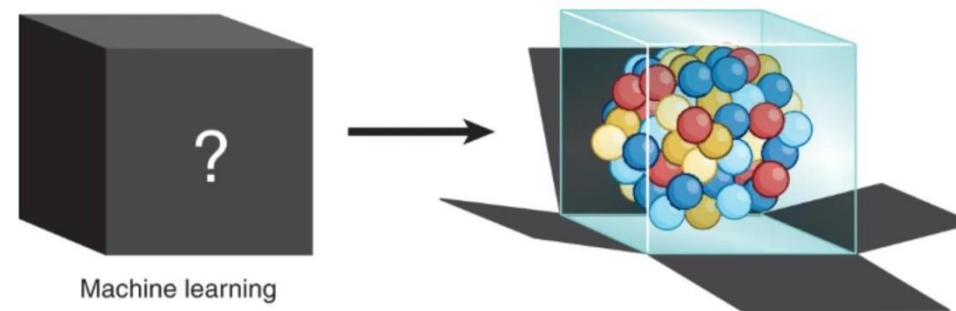
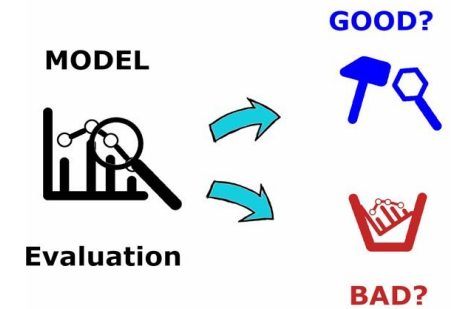
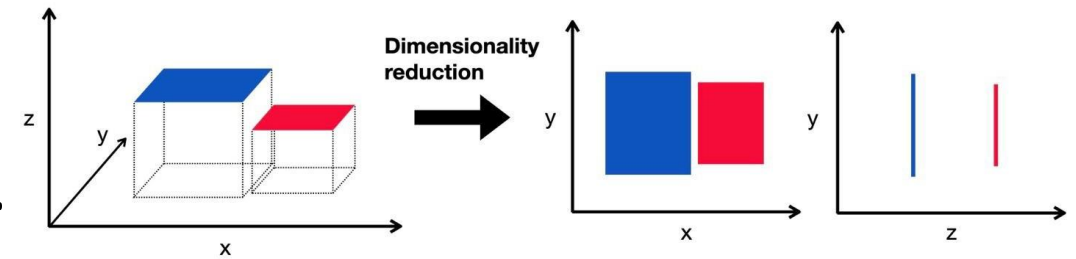
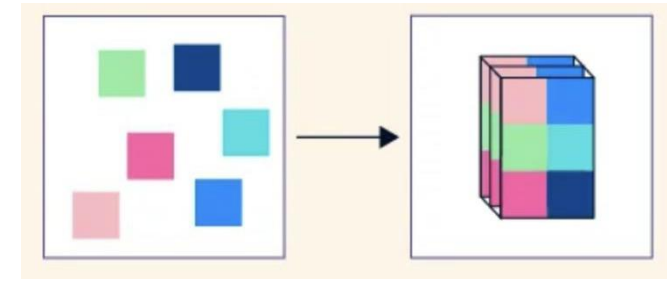
Text Representation

- Process of representing text data in a way that can be easily processed by machines
- Determines how the text data is transformed and processed by machine learning models
- Choice of text representation can significantly impact the performance of these models
- Essential to choose a representation that is appropriate for the task at hand



Importance of Text Representation

- Enables Computers to Understand Text
- Dimensionality reduction
- Improves Performance in NLP Applications.
- Captures Meaning and Relationships.
- Transforms Unstructured Text into Structured Features



In NLP,
an ordinary algorithm with strong text representation
often outperforms
a top algorithm applied with weak text representation.

Text Representation Techniques (most common)

Discreet Representation

- 1-hot encoding
- N-grams
- Bag-of-words
- Vector semantics (tf-idf)

Distributed Representation

- Word Embeddings
 - Word2vec
 - GloVe
 - Fast Text
- Contextualized Word Embeddings
 - BERT
 - GPT
 - ELMo

Basic Vectorization approach

Vocabulary Corpus(V)

[dog, bites, man, eats, meat, food]

Documents Corpus

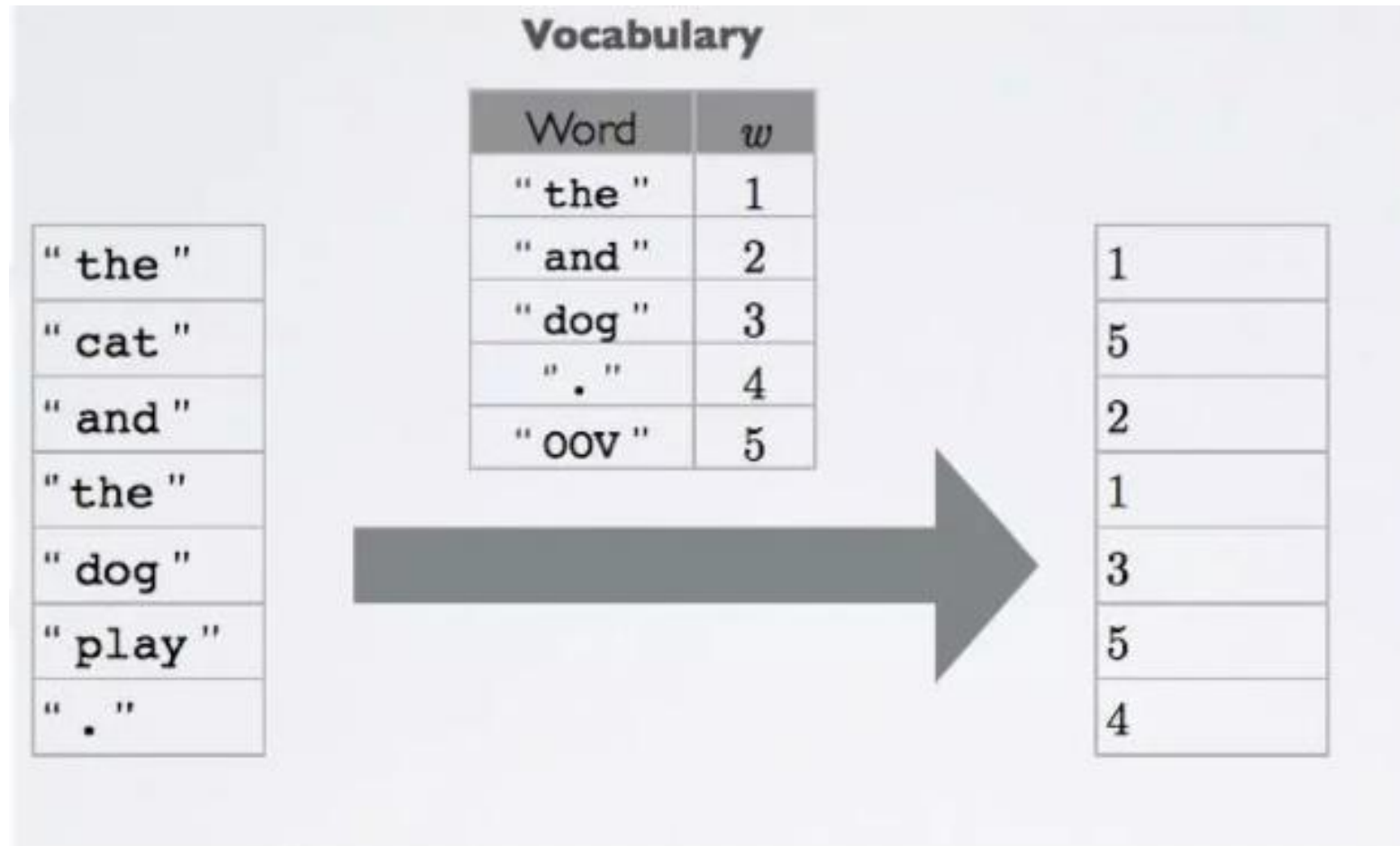
D1	Dog bites man.	→	[1, 1, 1, 0, 0, 0]
D2	Man bites dog.	→	[1, 1, 1, 0, 0, 0]
D3	Dog eats meat.	→	[1, 0, 0, 1, 1, 0]
D4	Man eats food.	→	[0, 0, 1, 1, 0, 1]

Just one of the many possible ways, we can make use of the vocabulary position based vector form.

1-hot encoding

1-hot Encoding

- Associate each unique word with an index in this vector.



1-hot Encoding

- To represent a unique word:
 - set the component of the vector to be 1, and zero out all of the other components

For example, in a 6-word Vocabulary corpus,

“dog” becomes $[1\ 0\ 0\ 0\ 0\ 0]$,

“bites” becomes $[0\ 1\ 0\ 0\ 0\ 0]$,

and so on.

Vocabulary Corpus(V)

[dog, bites, man, eats, meat, food]

A document like “dog bites man” is then a sequence of such vectors.



1-hot Encoding - Exercise

D1 Dog bites man.

D2 Man bites dog.

D3 Dog eats meat.

D4 Man eats food.

1-hot Encoding

- Is a binary representation method
- It only captures whether a word is present or not in a text, but not the frequency
- Size of the vector grows linearly with the size of the vocabulary - impractical for large vocabularies
- Useful for many text classification and information retrieval tasks, particularly those that involve small vocabularies and sparse data.

1-hot Encoding

- **Advantages of one-hot encoding:**
 - Easy to understand & implement
- **Disadvantages of one-hot encoding:**
 - Explosion in feature space if number of categories are very high
 - The vector representation cannot determine or measure relationship between different words
 - Cannot measure importance of a word in a sentence but understand mere presence/absence of a word in a sentence
 - High dimensional sparse matrix representation can be memory & computationally expensive
 - It can't handle unseen words (OOV) without retraining.

Bag of Words

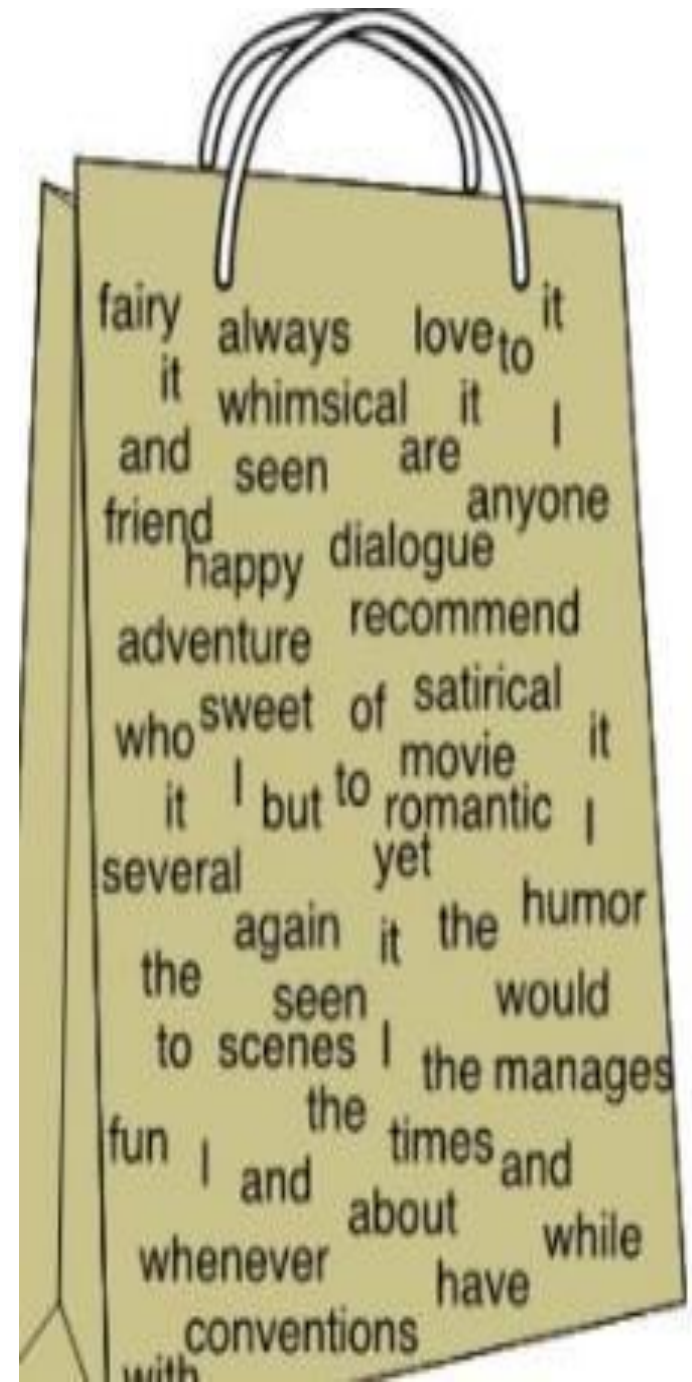
Bag of Words

- Represents a piece of **text as a bag** of its individual words
- Ignore their grammatical structure and word order
- Keep track of their frequency of occurrence

D1	Dog bites man.	→	[1, 1, 1, 0, 0, 0]
D2	Man bites dog.	→	[1, 1, 1, 0, 0, 0]
D3	Dog eats meat.	→	[1, 0, 0, 1, 1, 0]
D4	Man eats food.	→	[0, 0, 1, 1, 0, 1]

Bag of Words

- Typically involves the following steps:
 - **Tokenization:** The text is first split into individual words or tokens using a tokenizer, which may involve removing stop words and punctuation.
 - **Vocabulary creation:** A vocabulary of all unique words (tokens) in the text corpus is created, and each word is assigned a unique index.
 - **Vectorization:** Each text document is represented as a vector of the same length as the vocabulary size. The value of each element in the vector represents the frequency of the corresponding word in the text document.



Bag of Words

"The quick brown fox."

"Jumped over the lazy dog."

"The quick brown fox jumped over the lazy dog."

- The bag-of-words representation of these sentences might look like this:

Vocabulary: [The, quick, brown, fox, jumped, over, lazy, dog]

Sentence 1: [1, 1, 1, 1, 0, 0, 0, 0]

Sentence 2: [1, 0, 0, 0, 1, 1, 1, 1]

Sentence 3: [2, 1, 1, 1, 1, 1, 1, 1]

Bag of Words - Exercise

S1: “The cat sat on the hat”

S2: “The dog ate the cat and the hat”

Use the Bag-of-Words model to represent the sentences as an unordered set of words.

Disregard grammar and word order but consider word frequency.

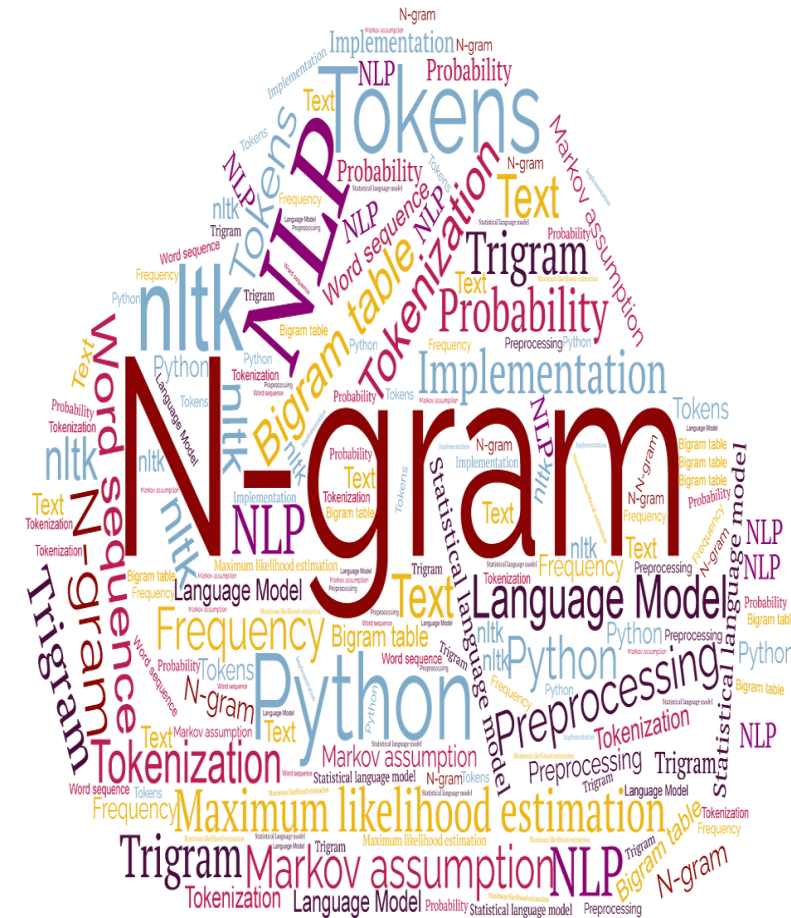
Bag of Words

- Useful for many NLP and IR tasks, such as text classification, document clustering, and information retrieval.
- **Advantages**
 - Easy to implement
 - Can get the frequency of each word
 - The size of the vector is the size of the dictionary.
- **Disadvantages**
 - Do not extract any semantic meaning from the words.
 - This method also loses the positional information of a word.
 - High-frequency words do not necessarily have the highest weightage.
 - Fails on unseen (OOV) words.
 - Ignores word order, so different sentences can look identical.

N-gram Models

N-gram Models

- Involves breaking down a sequence of words into contiguous sequences of n words.
- In other words, an n -gram is a sequence of n consecutive words
- N -gram language models estimate the probability of the last word given the previous words
- Useful for tasks, such as text classification, language modeling, and information retrieval.
- Capture local patterns and dependencies between adjacent words in a text.
- Use of n -grams requires careful consideration of the choice of n .
 - Using small values of n can lead to oversimplification and loss of information,
 - Using large values of n can result in sparse and high-dimensional representations, which can be computationally expensive and prone to overfitting



N-gram Models

"The quick brown fox jumps over the lazy dog."

- **Unigrams (n=1):** ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog."]
- **Bigrams (n=2):** ["The quick", "quick brown", "brown fox", "fox jumps", "jumps over", "over the", "the lazy", "lazy dog."]
- **Trigrams (n=3):** ["The quick brown", "quick brown fox", "brown fox jumps", "fox jumps over", "jumps over the", "over the lazy", "the lazy dog."]
- **Four-grams (n=4):** ["The quick brown fox", "quick brown fox jumps", "brown fox jumps over", "fox jumps over the", "jumps over the lazy", "over the lazy dog."]

Bag of N-grams

- Bag-of-n-grams (BoN) improves on earlier methods by capturing word order through n-grams—contiguous word sequences.
- Each document is represented by a vector of n-gram counts.
- BoW is a special case of BoN with $n=1$.
- Higher n captures more context but increases sparsity.
- For example:
 - Bag of N-grams when $N=2$, it becomes a bigram (2-gram) model
 - Text is represented by counts of word pairs (e.g., “dog bites”).

Bag of N-grams - Exercise

D1 Dog bites man.

D2 Man bites dog.

D3 Dog eats meat.

D4 Man eats food.

Create a Bag of N-grams considering a bi-gram model.

Bag of N-grams - Exercise

"Machine learning is a powerful tool for data analysis and predictions.

It involves training a model on historical data to make accurate predictions on new, unseen data."

Create a Bag of N-grams considering a trigram model.

Bag of N-grams

- **Advantages**

- It captures some context and word-order information in the form of n-grams
- It makes similar documents closer in vector space.

- **Disadvantages**

- Higher n increases dimensionality and sparsity, costing us computationally.
- Still provides no way to address unseen (OOV) words.

TF-IDF

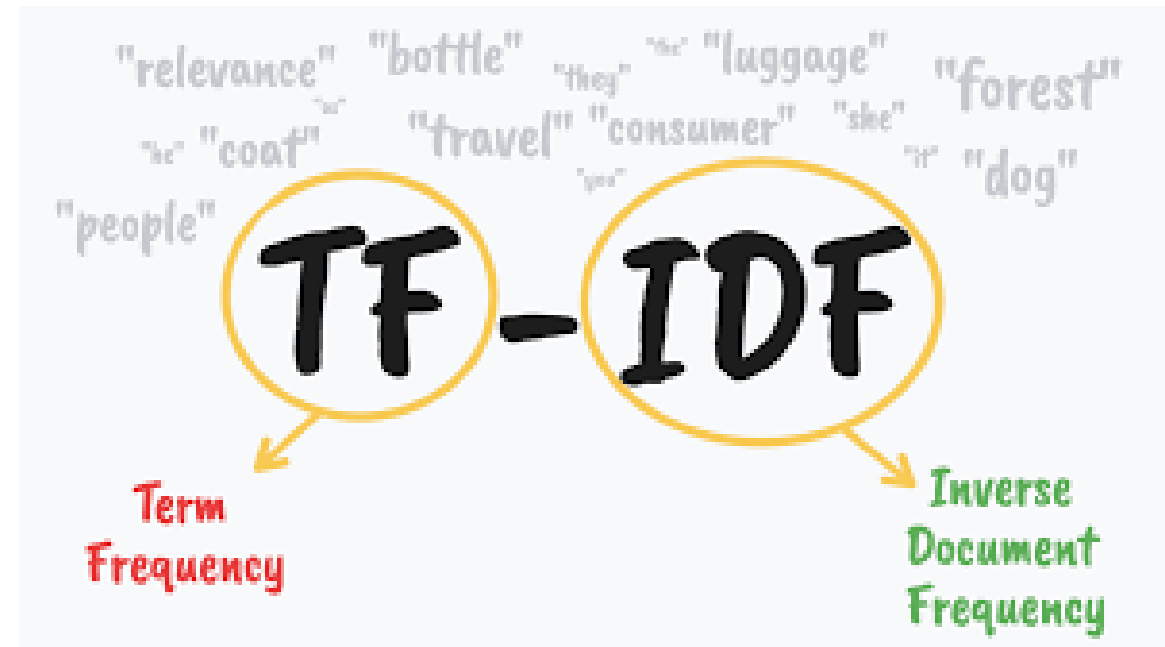
TF-IDF

- Term Frequency-Inverse Document Frequency
- Addresses the limitations of the bag-of-words representation
- Takes into account both the

frequency of occurrence of a
word in a document (TF)

and

its inverse frequency in the entire
corpus (IDF).



TF-IDF

- The TF-IDF weight for a word w in a document d is calculated as follows:

$$\text{TF-IDF}(w,d) = \text{TF}(w,d) * \text{IDF}(w)$$

Where,

- $\text{TF}(w,d)$ = Number of times term w appears in document d
- $\text{IDF}(w) = \log(N / n)$,

where N is the total number of documents in the corpus, and n is the number of documents containing word w .

- The IDF component penalizes words that occur frequently in the corpus and assigns a higher weight to words that are rare but highly relevant to a specific document.
- The logarithmic scaling of the IDF value helps to reduce the impact of very rare or very common words.

TF-IDF

- Unlike previous methods, it doesn't treat all words equally.
- The more a word appears in a document but not in others, the more important it is for that document.
- **TF-IDF** assigns importance to words based on **how often they appear in a document** (TF) and **how rare they are across the corpus** (IDF).

TF-IDF

- IDF (Inverse Document Frequency) **reduces the weight of common terms** (like stop words) and **increases the weight of rare ones** across the corpus. It adjusts TF to reflect a term's importance globally.
- TF (Term Frequency) **measures how often a term appears** in a document, **normalized to** account for document **length differences**.

TF-IDF

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

$$\textit{TF-IDF score} = \textit{TF} * \textit{IDF}.$$

TF-IDF

Document 1: "The quick brown fox jumps over the lazy dog."

Document 2: "The brown fox is quick."

Document 3: "The lazy dog is slow."

- The TF-IDF weight for each word in each sentence might look like this:

Word:	The	quick	brown	fox	jumps	over	lazy	dog	is	slow
Sentence 1:	[0,	0.176,	0.176,	0.176,	1.099,	1.099,	0.176,	0.176,	0,	0]
Sentence 2:	[0,	0.176,	0.176,	0.176,	0,	0,	0,	0,	0.176,	0]
Sentence 3:	[0,	0,	0,	0,	0,	0,	0.176,	0.176,	0.176,	1.099]

Here, the TF calculation is done without the normalization (TF = Number of occurrences of term t in document d)

TF-IDF: Exercise

Consider the following collection of documents:

Document 1: "Natural language processing is a branch of artificial intelligence."

Document 2: "Machine learning algorithms analyse data to make predictions."

Document 3: "Word embeddings capture semantic relationships in language."

Calculate the TF-IDF value for the word "language" in each document.

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

$$\text{TF-IDF score} = \text{TF} * \text{IDF}$$

TF-IDF

- **Advantages**

- Easy to understand and interpret.
- Penalizes both high and low-frequency words.

- **Disadvantages**

- We do not capture the positional information of a word.
- Semantic meanings of the words are not captured.
- Discrete Representations -> Treats words and n-grams as isolated units, missing inter-word relationships.
- Sparse, High-Dimensional Vectors -> Feature vectors grow with vocabulary size, are mostly zeros, and hurt learning efficiency.
- Inability to Handle OOV Words -> Struggles with words not seen during training.

Coding

1. One Hot Encoding:

"I love pizza",
"I love pasta",
"Cats are cute",
"Dogs are loyal",
"I love dogs"

- (i) Steps (Send as a Direct Message)
- (ii) Code (Upload to VLE) [Python Notebook]

2. Bag of Words:

I love cats

I love dogs

I love cats and dogs

Dogs love me too

Cats and dogs are pets

- (i) Steps (Send as a Direct Message)
- (ii) Code (Upload to VLE) [Python Notebook]

Limitations of Early Methods

While both BoW and TF-IDF are easy to implement and provide a reasonable baseline for some NLP tasks, they share significant limitations:

- **Lack of Context:** Both methods ignore the order of words, making it impossible to capture context, grammar, and word relationships.
- **High Dimensionality:** As the vocabulary grows, the representation size increases, leading to computational inefficiency.
- **Semantic Limitations:** These models do not capture the meaning or relationships between words. For example, “dog” and “puppy” would be treated as unrelated words despite their apparent semantic connection.

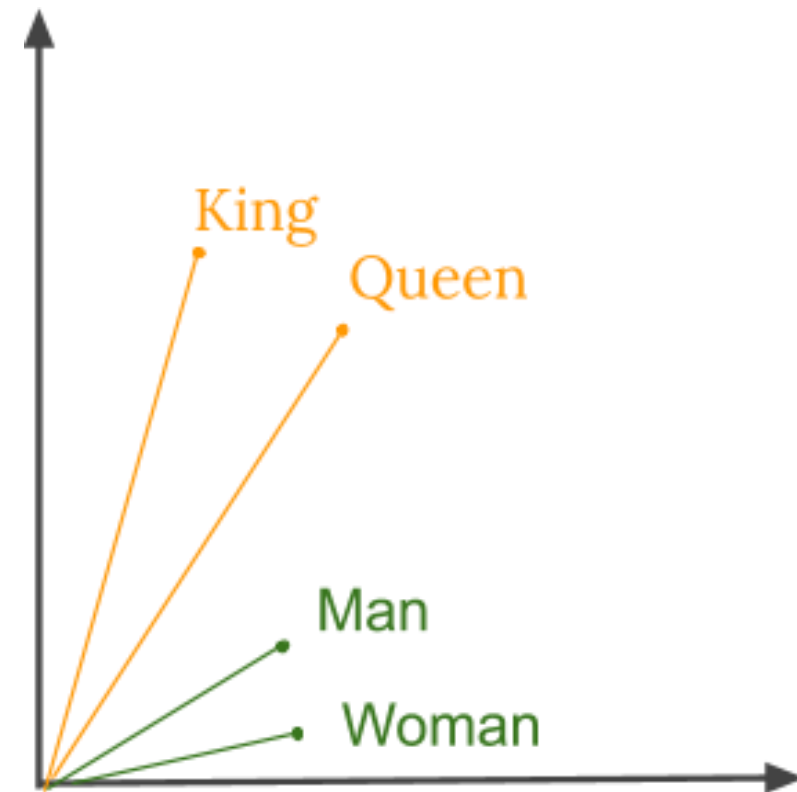
Word Embeddings

Word Embedding

- A real-valued vector that encodes the meaning of the word in such a way that the words that are closer in the vector space are expected to be similar in meaning.
- They are numerical vector representations of words or phrases, designed to capture the semantic and contextual meaning of language.
- These representations allow algorithms to understand the relationships between words and make better sense of textual data.

$$\text{KING} - \text{MAN} + \text{WOMAN} = \text{QUEEN}$$

- Methods:
 - Word2vec
 - GloVe
 - FastText
 - BERT
 - GPT



Example

king	[0.82, 0.15, 0.64, 0.33, 0.91]
------	--------------------------------

queen	[0.80, 0.18, 0.60, 0.35, 0.89]
-------	--------------------------------

man	[0.61, 0.12, 0.42, 0.20, 0.70]
-----	--------------------------------

woman	[0.60, 0.14, 0.45, 0.22, 0.68]
-------	--------------------------------

apple	[0.10, 0.88, 0.05, 0.91, 0.12]
-------	--------------------------------

Word Embedding

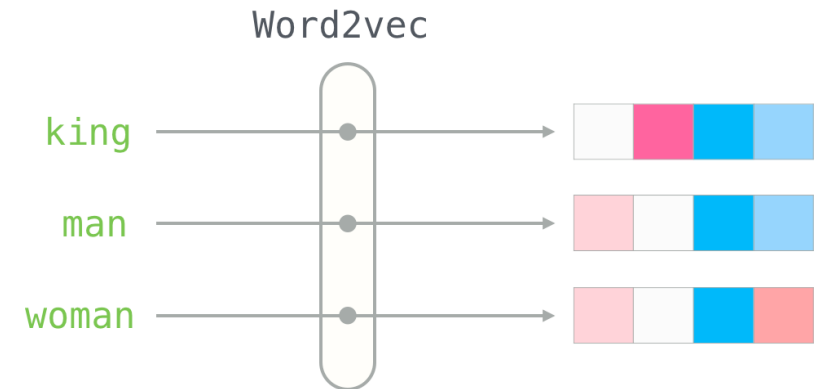
Key Features of Word Embeddings:

- **Dense Representation:** Unlike traditional methods like one-hot encoding which produce high-dimensional and sparse vectors, word embeddings are dense and typically of much lower dimensionality (e.g., 100-300 dimensions).
- **Semantic Similarity:** The vectors generated by word embeddings capture semantic meanings. Words that are semantically similar are located close to each other in the vector space.
- **Contextual Information:** Depending on the model, embeddings can capture the context in which words appear. Contextual embeddings like those from BERT or GPT capture the meaning of a word based on its surrounding words in a sentence.

Word Embedding – Word2vec

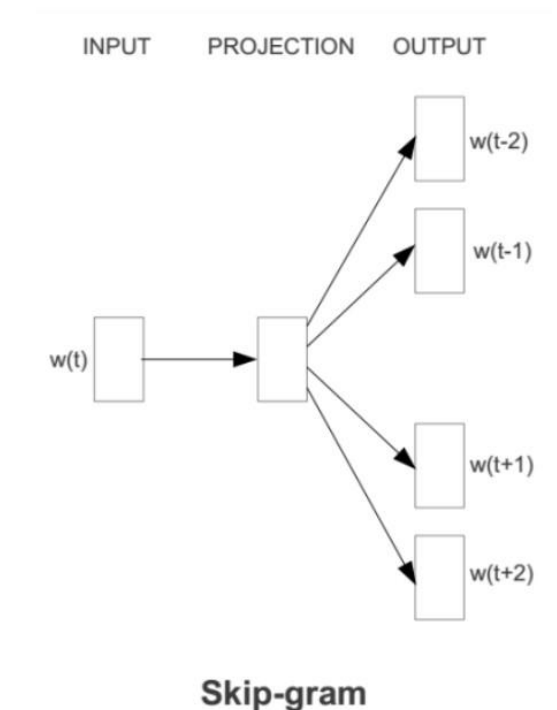
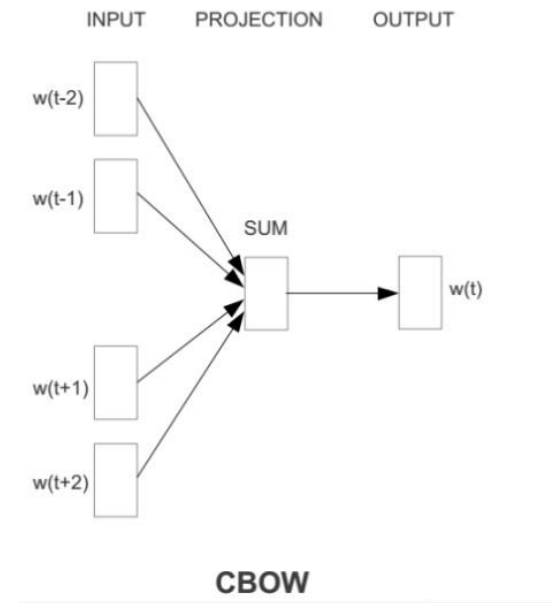
- Word2Vec is a popular model that learns word embeddings from large corpora of text.
- was originally proposed at Google by Mikolov et al.
- The word2vec objective function causes the word that occur in similar contexts to have similar embeddings.

- Example:
 - The **kid** said he would grow up to be superman.
 - The **child** said he would grow up to be superman.
- The **kid** and **child** will have similar word vectors due to similar context.



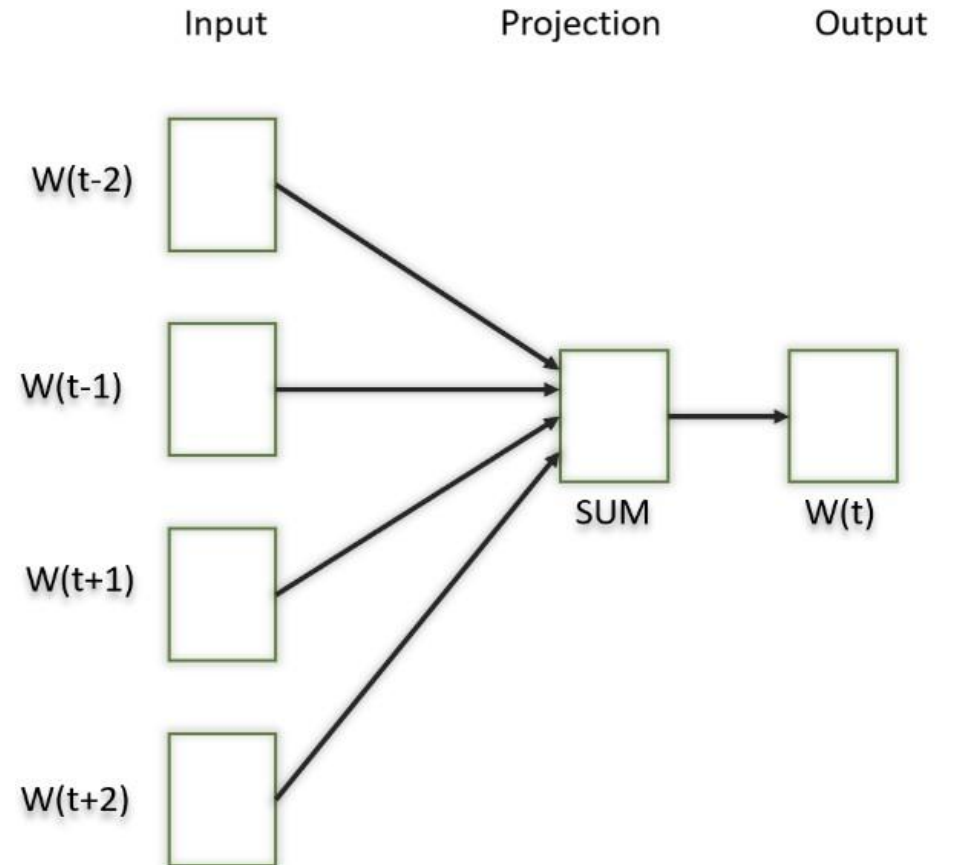
Word Embedding – Word2vec

- It is based on two techniques (Learning Models):
 - Continuous Bag of Words (CBOW)
 - Skip-gram.
- **CBOW**: use context words in a window to predict the middle word
- **Skip-gram**: use the middle word to predict surrounding ones in a window



Word Embedding – Word2vec

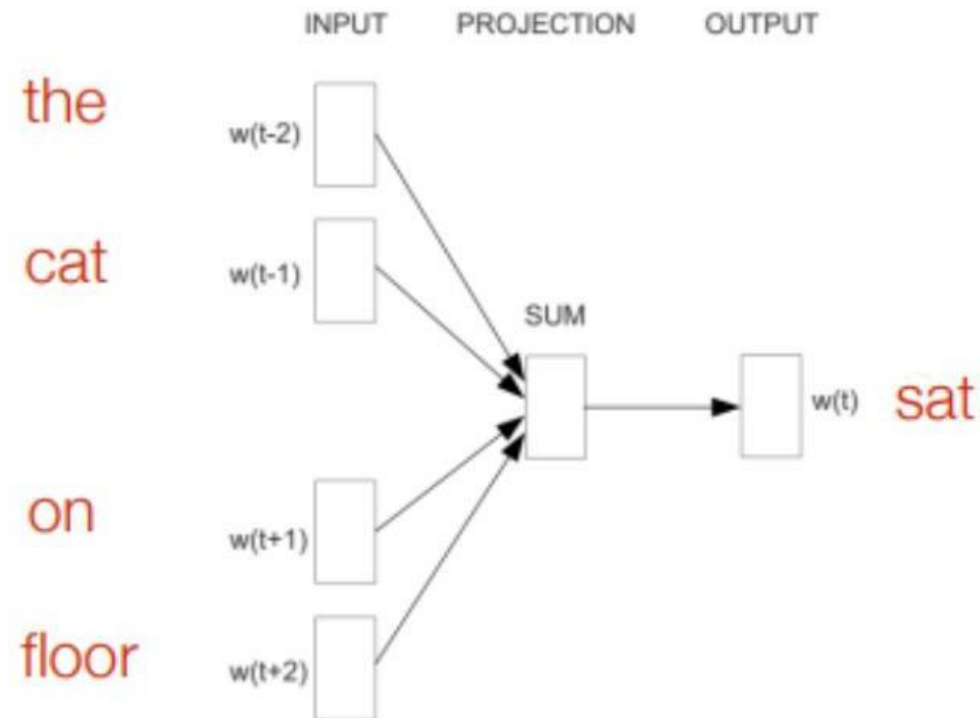
- Continuous Bag of Words (CBOW):
 - Use context words in a window to predict the middle word
 - Given a sequence of words in a context window, the model is trained to predict the target word at the center of the window
 - This technique uses the shallow 2-layer neural network to predict the probability of a word given the context.
 - A context can be a single word or a group of words.
 - Predict the target word from the context.



Word Embedding – Word2vec

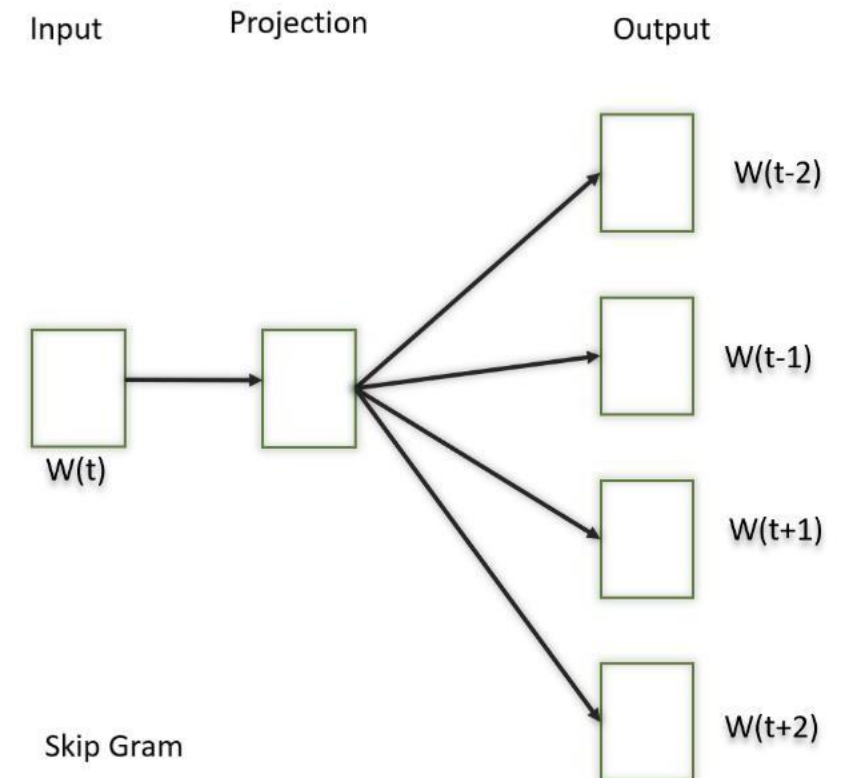
- CBOW

Example: “The cat sat on floor” (window size 2)



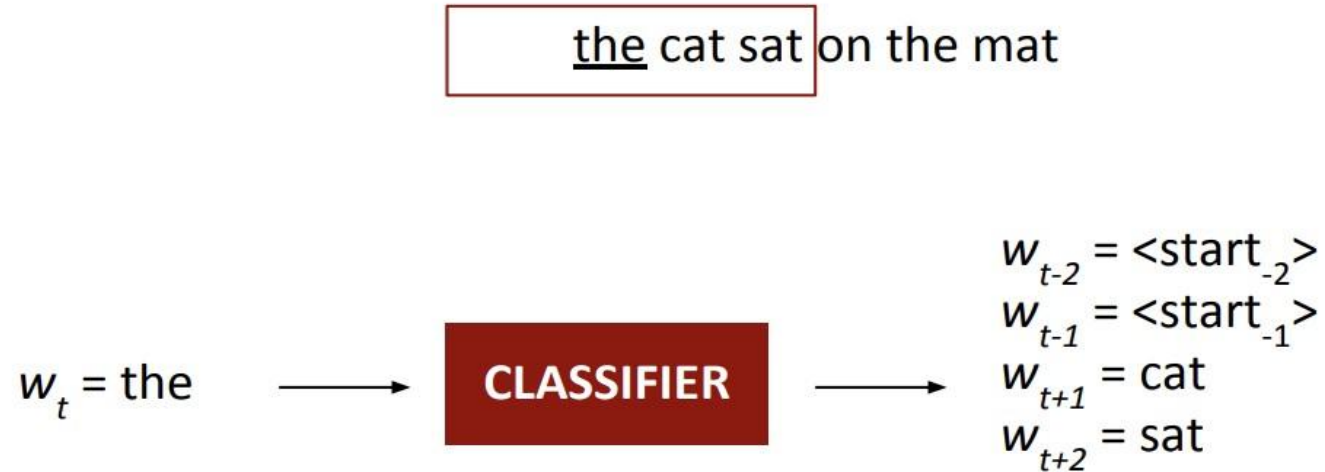
Word Embedding – Word2vec

- Skip-gram
 - Predict context words (words surrounding a target word) given a target word.
 - This is the opposite of the Continuous Bag of Words (CBOW) model.
 - It predicts words in a certain range before and after the current word.



Word Embedding – Word2vec

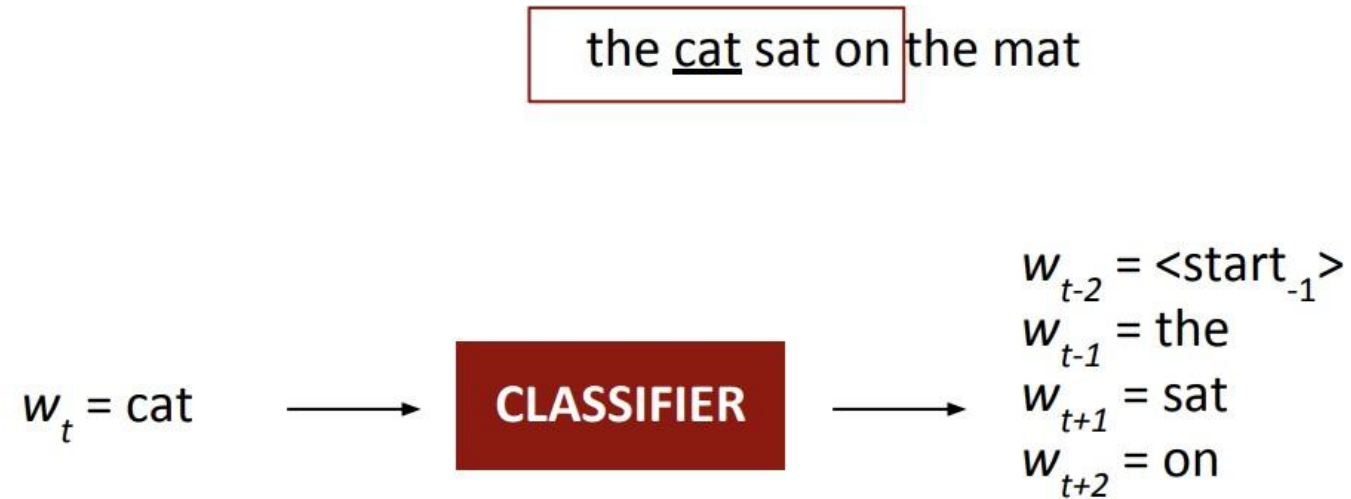
- Skip-gram: Example



context size = 2

Word Embedding – Word2vec

- Skip-gram: Example



context size = 2

Word Embedding – Word2vec

- Skip-gram: Example

the cat sat on the mat

$w_t = \text{sat}$



CLASSIFIER



$w_{t-2} = \text{the}$
 $w_{t-1} = \text{cat}$
 $w_{t+1} = \text{on}$
 $w_{t+2} = \text{the}$

context size = 2

Word Embedding – Word2vec

- Skip-gram: Example

the cat sat on the mat



context size = 2

Word Embedding – Word2vec

- Skip-gram: Example

the cat sat on the mat

$w_t = \text{the}$



CLASSIFIER



$w_{t-2} = \text{sat}$

$w_{t-1} = \text{on}$

$w_{t+1} = \text{mat}$

$w_{t+2} = \text{<end}_{+1}\text{>}$

context size = 2

Word Embedding – Word2vec

- Skip-gram: Example

the cat sat on the mat

$w_t = \text{mat}$



CLASSIFIER

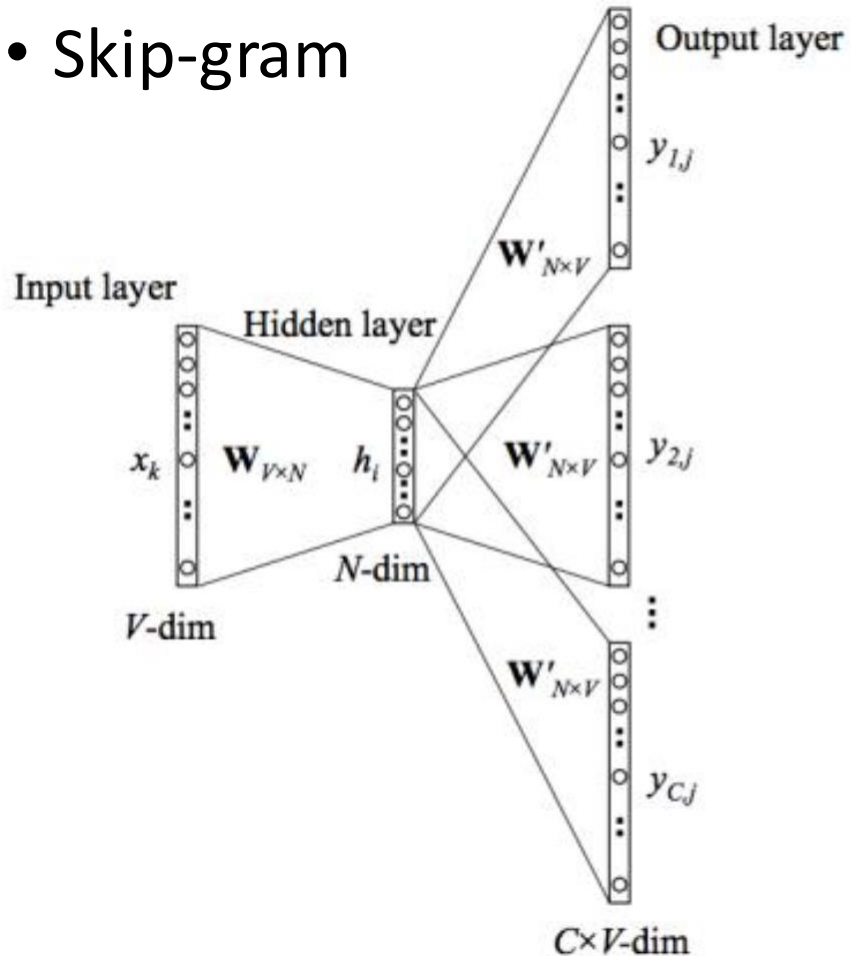


$w_{t-2} = \text{on}$
 $w_{t-1} = \text{the}$
 $w_{t+1} = \text{<end}_{+1}>$
 $w_{t+2} = \text{<end}_{+2}>$

context size = 2

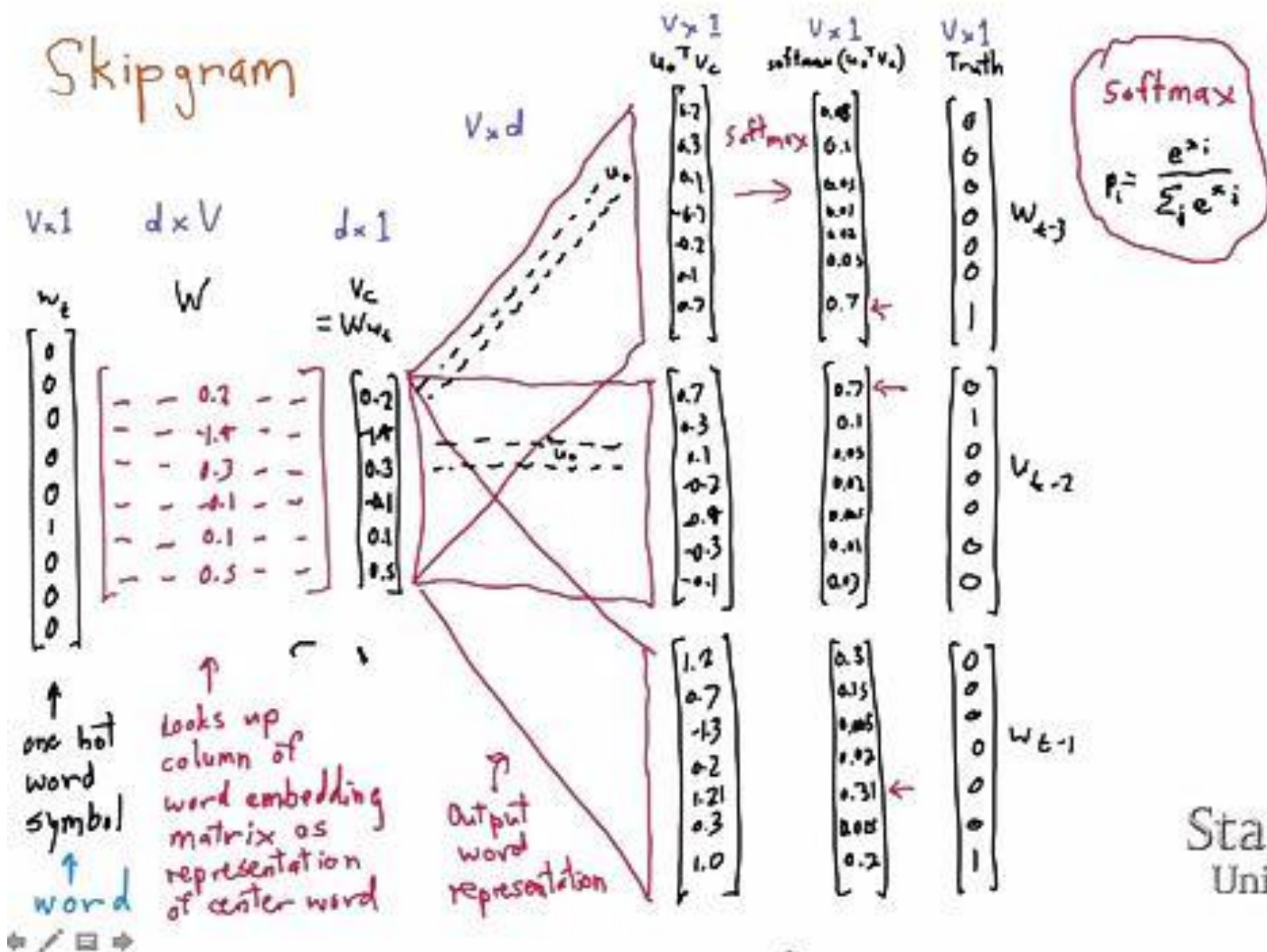
Word Embedding – Word2vec

- Skip-gram



Window Size	Text	Skip-grams
2	[The <u>wide</u> road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road <u>shimmered</u> in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot <u>sun</u>].	sun, the sun, hot

Skipgram



Word Embedding – Word2vec

- **Advantages:**

- Capable of capturing relationships between different words including their syntactic & semantic relationships
- The size of the embedding vector is small & flexible, unlike all the previous algorithms discussed where the size of embedding is proportional to vocabulary size
- Since its unsupervised, human effort in tagging the data is less

- **Disadvantages:**

- Word2Vec cannot handle out-of-vocabulary words well. It assigns a random vector representation for OOV words which can be suboptimal
- It relies on local information of language words. The semantic representation of a word relies only on its neighbours & can prove suboptimal
- Parameters for training on new languages cannot be shared. If you want to train word2vec in a new language, you have to start from scratch
- Requires a comparatively larger corpus for the network to converge (especially if using skip-gram)

Question

- Consider two sentences containing the same word in different contexts.
 - I went to the [bank] to deposit money
 - I sat on the river [bank]
- Explain how Word2Vec handles the representation of this word during training with a fixed window size.
- Discuss how the final embedding is affected by multiple occurrences of the same word.

Word Embedding – GloVe

- Global Vectors for word representation
- It tries to overcome the disadvantage of word2vec mentioned above by trying to learn both local & global statistics of a word to represent it
- Learns vectors of words using their co-occurrence statistics
- We take the corpus and iterate through it and get the co-occurrence of each word with other words in the corpus

Word Embedding – GloVe

- Example:

the cat drinks milk

the dog eats meat

the cat and dog are animals

Window Size = 3

Word $i \setminus j$	the	cat	drinks	milk	dog	eats	meat
the	0	2	1	1	2	1	1
cat	2	0	1	1	1	0	0
drinks	1	1	0	1	0	0	0
milk	1	1	1	0	0	0	0
dog	2	1	0	0	0	1	1
eats	1	0	0	0	1	0	1
meat	1	0	0	0	1	1	0
and	1	1	0	0	1	0	0
are	0	1	0	0	1	0	0
animals	0	1	0	0	1	0	0

Word Embedding – GloVe

- GloVe combines:
 - Local context (like Word2Vec)
 - Global statistics (co-occurrence matrix)

$$w_i^T w_j + b_i + b_j \approx \log(X_{ij})$$

Where:

- X_{ij} = co-occurrence count
- w_i, w_j = word vectors
- b_i, b_j = bias terms

$$w_i^T w_j$$

So:

- one vector represents center word
- one vector represents context word

Sentence: cat drinks milk

Input = “drinks”

Context = “cat”

$w_{\text{drinks}}(\text{input}) = [-0.3, 0.8, 0.1]$

$w_{\text{cat}}(\text{context}) = [0.1, 0.0, -0.3]$

Word Embedding – GloVe

- Exercise:

Consider two sentences -

I am a data science enthusiast

I am looking for a data science job

- Create the co-occurrence matrix involved in GloVe

Word Embedding – GloVe

- **Advantages**

- It tends to perform better than word2vec in analogy tasks
- It considers word pair to word pair relationship while constructing the vectors & hence tend to add more meaning to the vectors when compared to vectors constructed from word-word relationships
- GloVe is easier to parallelise compared to Word2Vec hence shorter training time

- **Disadvantages**

- Because it uses a co-occurrence matrix & global information, memory cost is more in GloVe compared to word2vec
- Similar to word2vec, it does not solve the problem of polysemous words since words & vectors have a one-to-one relationship

Word Embedding – FastText

- Developed by Facebook
- FastText extends Word2Vec by representing words as bags of character n-grams.
- This approach is particularly useful for handling out-of-vocabulary words and capturing morphological variations.
- FastText mitigates OOV problem by taking subword information into account.

Word Embedding – FastText

How FastText Works:

- FastText represents each word as a single entity and a collection of character-level n-grams (subword units). [character n-grams → vectors]
- For example,

the word “playing” might be broken down into subwords like <pl, pla, lay, ayi, yin, ing, ng>.
- These subwords are then used to build word embeddings, allowing FastText to generate embeddings for words not seen during training by combining known subword embeddings.
- Finally → a word = sum of subword vectors

Word Embedding – FastText

Advantages:

- **Handles rare and OOV words:** FastText can generate embeddings for rare or unseen words using subwords, making it more robust than Word2Vec and GloVe.
- **Better representation for morphologically rich languages:** FastText is particularly useful for languages with complex word forms, such as German or Finnish, where words are often built from many morphemes.

Benefits of Word Embeddings Over Early Methods

Word embeddings represent a significant improvement over early methods like BoW and TF-IDF:

- **Context and Meaning:** Word embeddings can capture semantic relationships between words, unlike the bag-of-words approach, which treats all words as independent.
- **Low Dimensionality:** Word embeddings are dense and low-dimensional (typically 100-300 dimensions), making them more efficient to store and compute than sparse vectors produced by BoW or TF-IDF.
- **Transferability:** Pre-trained word embeddings (e.g., those trained on large datasets like Wikipedia) can be transferred to new NLP tasks, providing a strong foundation for various applications without training from scratch.

Contextualized Word Embeddings

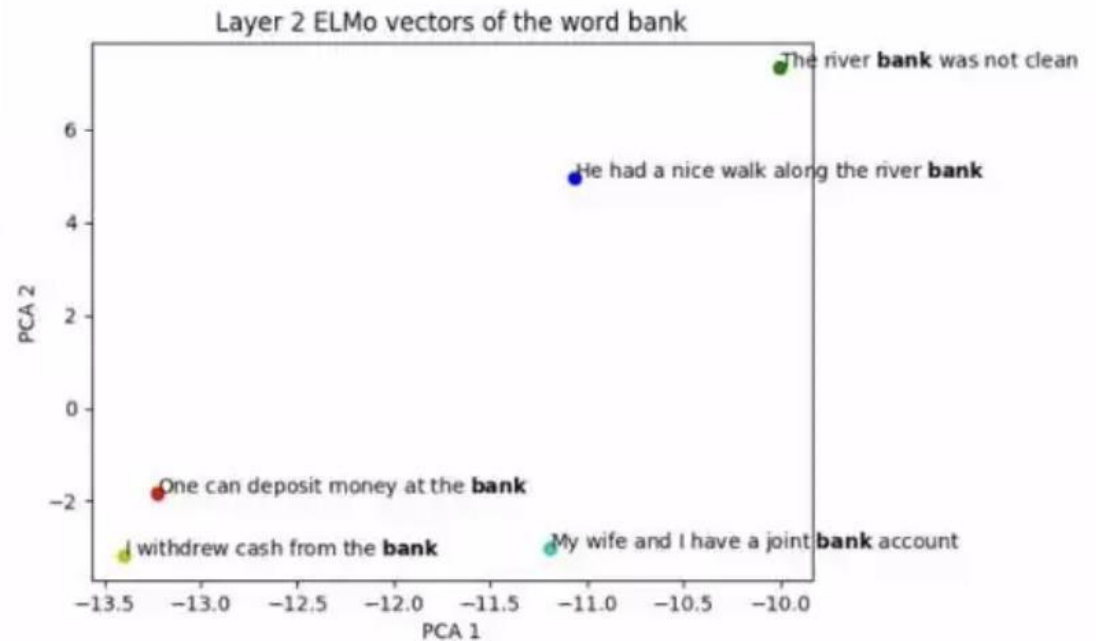
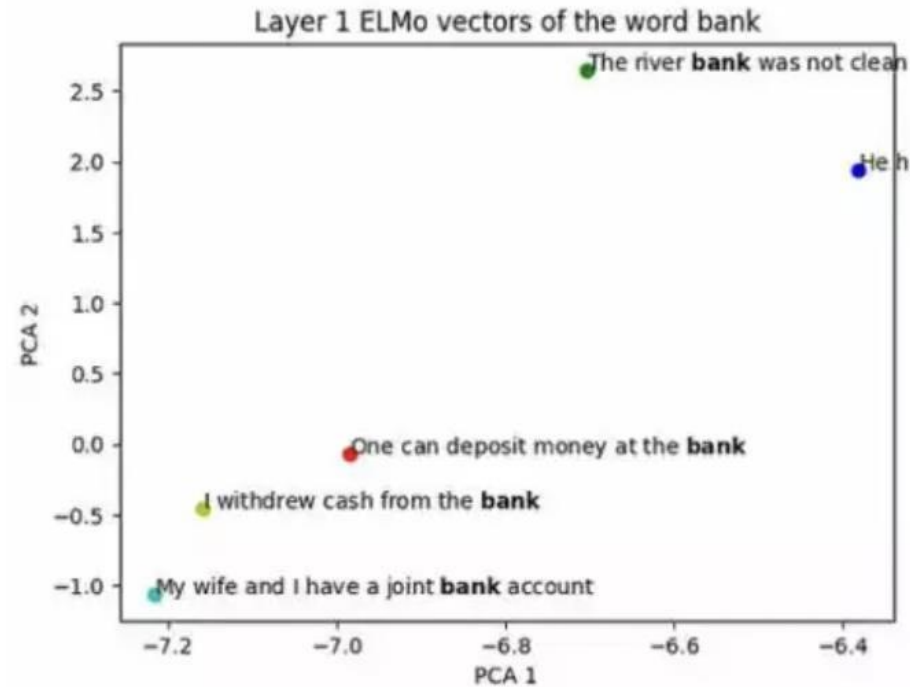
Contextualized Word Embeddings

- While traditional word embeddings like Word2Vec, GloVe, and FastText revolutionized text representation by capturing semantic relationships between words, they suffer from a significant limitation: **static representations**.
- These embeddings assign a single vector to each word, regardless of its context, meaning that words with multiple meanings (like “bank” in “financial bank” vs “river bank”) are treated the same.

Word Embedding - ELMo

- ELMo (Embedding from language models)
 - It tries to solve the disadvantages of word2vec & GloVe by having a many-to-one relationship between vector representation & the word it represents.
 - i.e it incorporates the context & changes the vector representation of the word accordingly.
 - Unlike static word embeddings, ELMo represents words differently based on their usage in a sentence.

Word Embedding - ELMo



Word Embedding - ELMo

How ELMo Works:

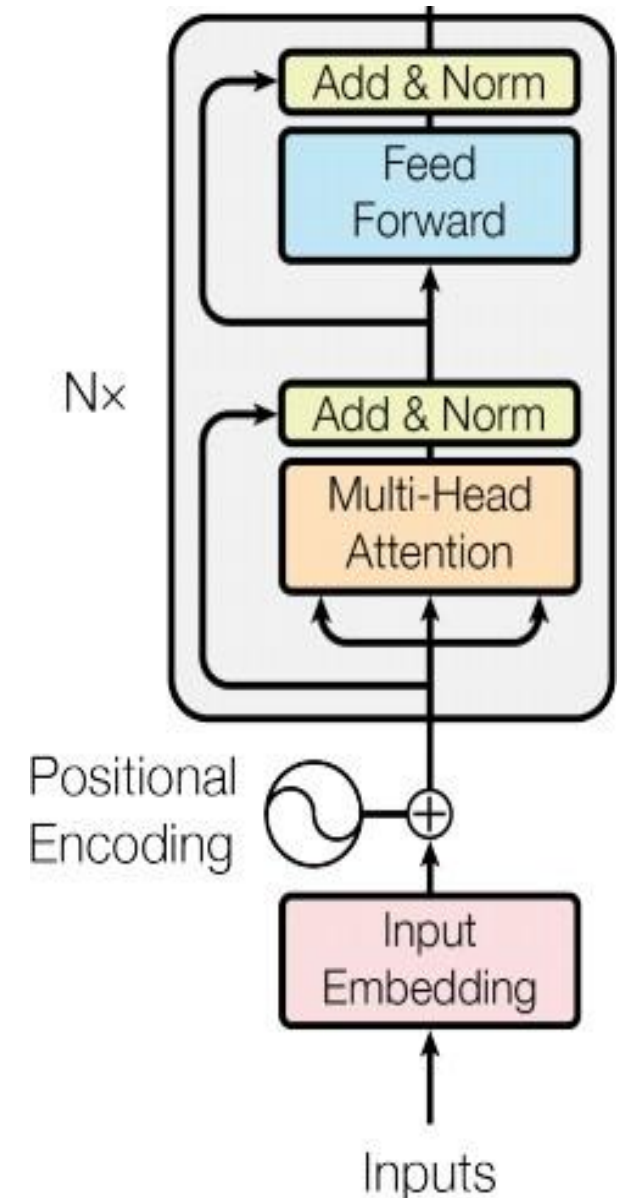
- ELMo uses a deep **bidirectional LSTM** (Long Short-Term Memory) network trained on large text corpora. **It generates word representations by looking at the entire sentence**, considering both the words before and after each target word.
- Instead of assigning a single vector per word, ELMo produces **dynamic embeddings** that change depending on the context.

For example,

the representation of “apple” in “**I ate an apple**” will differ from its representation in “**Apple announced a new product.**”

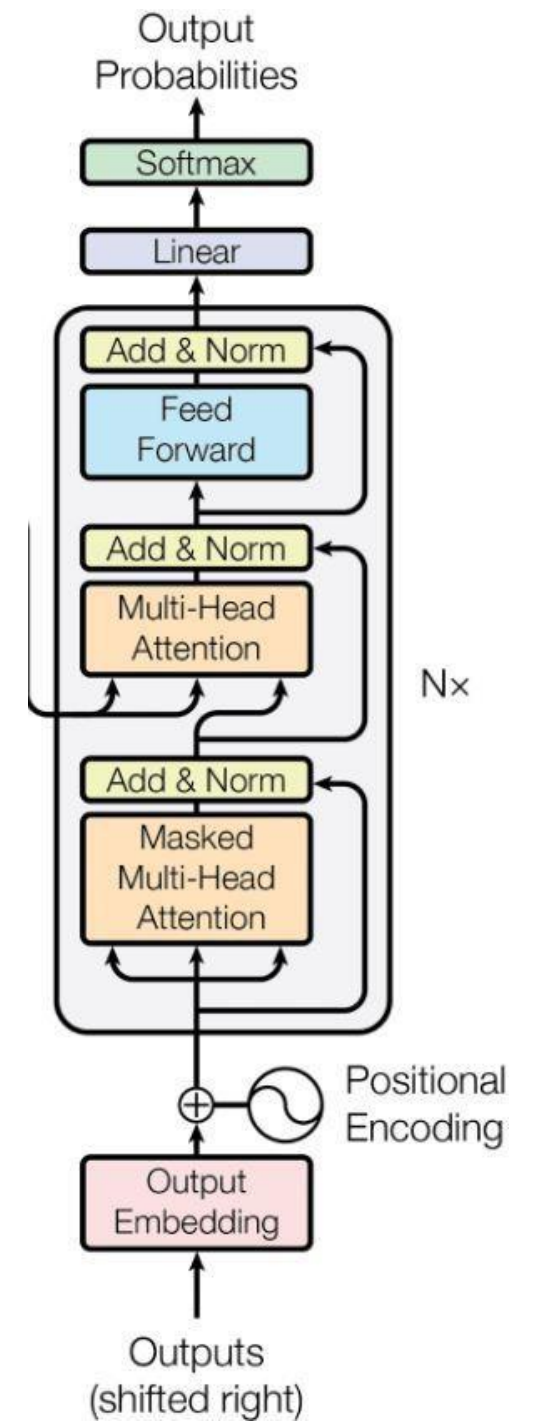
Word Embedding - BERT

- BERT (Bidirectional Encoder Representations from Transformers)
 - BERT is a transformer-based model that learns contextualized embeddings for words.
 - It considers the entire context of a word by considering both left and right contexts, resulting in embeddings that capture rich contextual information.



Word Embedding - GPT

- GPT (Generative Pretrained Transformer)
 - Focuses on generative pre-training and fine-tuning for specific tasks.
 - Uses transformer architecture to create context-aware representations.



Why Do Contextualized Embeddings Matter?

Contextualized word embeddings address several critical issues that static embeddings couldn't handle:

- **Disambiguation:** Words with multiple meanings (e.g., “bank”) are disambiguated based on the context in which they appear.
- **Context-specific nuances:** Subtle changes in meaning, tone, or intent are captured, leading to more accurate representations for tasks like sentiment analysis, question answering, and machine translation.
- **Performance on complex NLP tasks:** These embeddings have enabled significant improvements across various NLP tasks, including named entity recognition (NER), machine translation, and conversational AI, by dynamically adapting to context

Beyond BERT and GPT

Since BERT and GPT, numerous models have further advanced the field of contextualized embeddings:

- **RoBERTa**: An optimized version of BERT with improved pretraining methods, yielding better performance.
- **ALBERT**: A more lightweight version of BERT that reduces the model size while maintaining accuracy.
- **T5 (Text-to-Text Transfer Transformer)**: A model that frames every NLP task as a text-generation problem, using a unified approach to tasks like summarization, translation, and question answering.
- **GPT-4**: OpenAI's latest iteration in the GPT series is capable of even more sophisticated language generation and understanding.
- ...